

Workflows Lead.

From stub endpoints to a full transactional backend, in five days.

"The plumbing that turns agents into a product."

A complete, paste-ready Antigravity build guide for the Backend Workflows track of the Bulao project — AI Seekho 2026.

Every prompt is self-contained. Every file path, dependency, schema, and convention is spelled out. Antigravity will not need to guess.

How to use this document:

1. Read 'Your role' and 'System context' first (sections 01-02).
2. On each build day, paste the day's prompt into Antigravity.
3. Verify with the day's checklist before moving on.
4. Reference the appendices when Antigravity asks for clarification.

01 · YOUR ROLE

You own backend workflows.

You are the **Workflows Lead** on the Bulao build team. There are four teammates total — Product & Urdu, Mobile, Backend, Infrastructure & Trace. This document is yours alone. Each teammate has their own pack. Do not try to do the others' work; coordinate handoffs through WhatsApp + a shared Drive.

What you own

- FastAPI `/orchestrate`, `/book`, `/dispute`, `/health`, `/rating` endpoints
- Booking Agent — confirmation message, receipt PDF, lifecycle init
- Follow-up & Dispute Agent — reminders, dispute classification + resolution
- Cloud Scheduler triggers for the Follow-up Agent
- Firestore persistence — bookings collection + subcollections
- The service-quality lifecycle state machine
- Cloud Run deployable Docker image + `scripts/deploy.sh`

What you do *not* own

- Intent, Discovery, Ranking, Pricing agents (Backend Agents owns these)
- Flutter / mobile code
- Cloud Run console and Secret Manager setup (Infra owns)

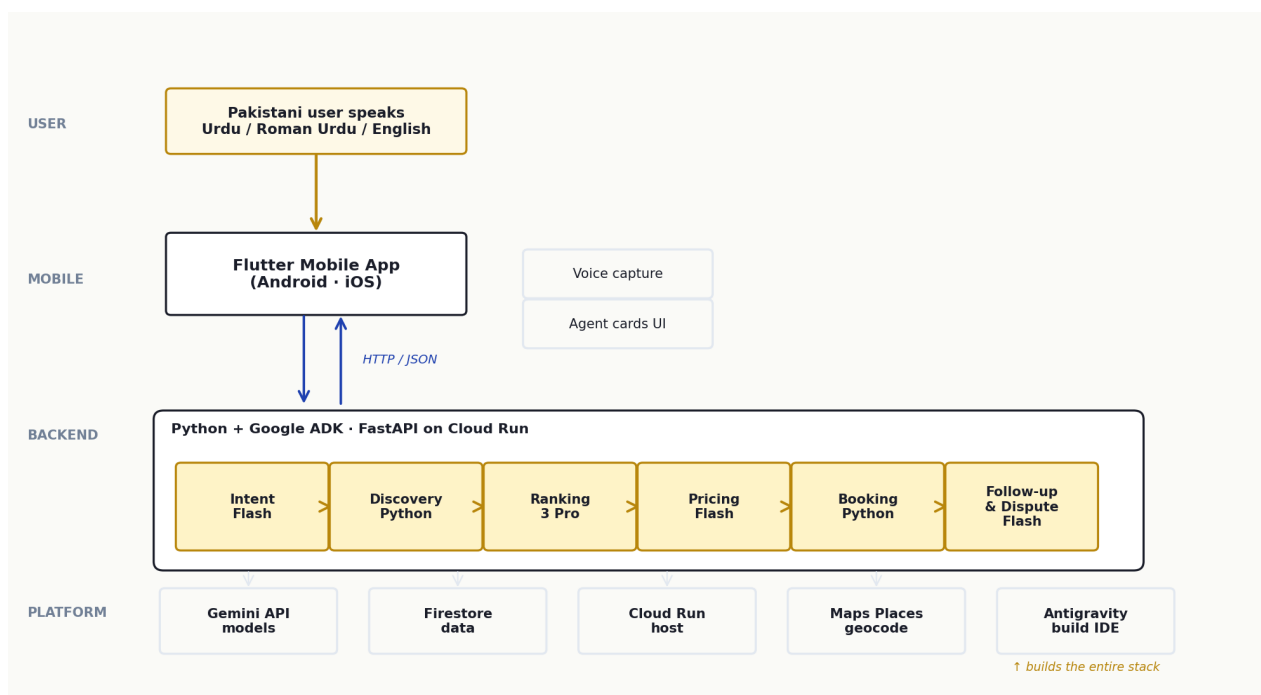
If you find yourself doing work outside the 'what you own' list, stop and ask in the team WhatsApp. Five days is short. Stay in your lane and move fast.

02 · SYSTEM CONTEXT

What Bulao is, end to end.

Bulao is a voice-first, Urdu-native AI service orchestrator for Pakistan's informal economy — plumbers, electricians, AC technicians, beauticians, tutors. A user presses a big button, speaks in Urdu / Roman Urdu / English, and within ninety seconds watches **six AI agents** extract their intent, find providers, rank them, produce a transparent price quote, book a slot, and follow up through completion — including fair dispute resolution when something goes wrong.

We are building this for the AI Seekho 2026 Antigravity Hackathon (Challenge 2). Five days, four teammates, one demo on May 20. Regional finals May 25-26, finals June 7 in Islamabad. Prize pool PKR 2.5M.



Three layers: a Flutter mobile app captures voice and renders six AgentCards; a Python backend on Cloud Run runs the six-agent pipeline using Google ADK and Gemini; Antigravity is the IDE that builds the stack — not part of the runtime, but the source of the 25%-weighted Agent Trace deliverable.

The six agents

Every teammate must know what each agent does, what it takes in, and what it produces. This is the shared mental model.



Your relationship to the six agents

You build the **execution layer** — the plumbing that turns agent outputs into real transactions. The Booking Agent generates the confirmation message and receipt PDF and writes to Firestore. The Follow-up & Dispute Agent fires on a Cloud Scheduler timer. You also own the FastAPI orchestrator that sequences all six agents, the /book and /dispute endpoints, and the Docker image that Infra deploys to Cloud Run.

Key handoffs you must respect

FROM	Day	TO	Deliverable
Backend Agents	1 evening	You	Scaffold at backend/ — poetry run uvicorn working
You	1 evening	Mobile+Infra	Stub /orchestrate, /book, /dispute with valid JSON shapes
Backend Agents	2 evening	You	Real intent + discovery modules importable
Backend Agents	3 noon	You	Real ranking + pricing modules importable
You	3 evening	Mobile	Full /orchestrate incl. Ranking + Pricing + Booking
You	4 evening	Mobile	Dispute path — rating < 3 triggers /dispute
You	5 morning	Infra	Docker image v1.0.0 + deploy.sh tested

03 · DAY 1 · BEFORE MAY 15

Day 0 (May 14) · setup

Same environment as Backend Agents: Python 3.11, Poetry, Docker, gcloud, AntigraVity. The Backend Agents teammate will scaffold the repo on Day 1 morning; you pull their commit by 6pm and start building on top.

Setup · install everything you need

This is not an AntigraVity prompt — do these by hand before May 15. AntigraVity cannot install software on your laptop for you.

PROMPT · LOCAL ENVIRONMENT CHECKLIST (RUN YOURSELF)

Complete these on your laptop by end of Day 0 (May 14):

1. Install Python 3.11 (NOT 3.12; google-adk's most stable line is on 3.11).
 - macOS: `brew install python@3.11`
 - Linux: `sudo apt install python3.11 python3.11-venv python3.11-dev`
 - Windows: download from python.org, check "Add to PATH"
2. Install Poetry (Python dependency manager):
 - `curl -sSL https://install.python-poetry.org | python3 -`
 - `poetry --version` # should show `>= 1.7`
3. Install Docker Desktop and verify it runs:
 - `docker --version`
 - `docker run --rm hello-world`
4. Install gcloud CLI:
 - <https://cloud.google.com/sdk/install>
 - `gcloud auth login` (use the same email as the team's GCP account)
 - `gcloud config set project bulao-hackathon`
 - `gcloud auth application-default login`
5. Install Google AntigraVity (<https://antigravity.google/download>).
 - Launch, sign in with the same Gmail.
 - Settings -> Agent panel:
 - Mode = Agent-assisted development
 - Artifact Review Policy = Asks for Review
 - Terminal Command Auto Execution = Request Review
 - Enable Terminal Sandbox = ON
 - Default model = Gemini 3 Pro
6. Get your Gemini API key from <https://aistudio.google.com> -> Get API key.
Create a key in a project named bulao-hackathon. Copy it; you will paste it into Secret Manager (Infra teammate will do this) and into your local .env file.

7. Clone the team repo (Infra will share the URL).

```
git clone
```

```
cd bulao
```

ls # you should see mobile/ backend/ docs/ at minimum

If anything fails, post in WhatsApp BEFORE Day 1 starts. Don't burn build hours on environment setup.

End-of-day verification

- `python3.11 --version` shows 3.11.x.
- `poetry --version` shows ≥ 1.7 .
- `docker run --rm hello-world` works.
- `gcloud projects list` shows bulao-hackathon.
- Antigravity launches; you can spawn a new Manager View workspace.
- You have your Gemini API key saved in your password manager.
- The team repo is cloned and backend/ directory exists.

04 · DAY 2 · MAY 15

Day 1 · scaffold coordination + stub endpoints

Backend Agents own the Day 1 scaffold. Your Day 1 job: pull their evening commit, verify you can run the app, and add stub routing for your endpoints so Mobile has valid URL contracts to wire against.

The scaffolding prompt (paste in Antigravity Manager View)

This single prompt builds the entire backend skeleton. Antigravity will produce a Plan Artifact first — review every file before approving. The prompt is self-contained: file paths, dependencies, schemas, and verification commands are all spelled out.

PROMPT · DAY 1 — SCAFFOLD THE BACKEND

ROLE: You are setting up the backend repository for "Bulao" - a six-agent voice-first Urdu service-orchestrator for Pakistan, built for AI Seekho 2026 Antigravity Hackathon.

WHAT TO BUILD TODAY: A scaffolded FastAPI backend with 6 placeholder agents, a working /orchestrate endpoint returning hardcoded mock JSON, a /health endpoint, a Dockerfile, and a passing test suite. Deployable to Cloud Run by end of day.

EXACT DIRECTORY STRUCTURE (create every file):


```
backend/  
README.md  
pyproject.toml  
Dockerfile  
.env.example  
.gitignore  
app/  
__init__.py  
main.py  
config.py  
orchestrator.py  
models.py  
agents/  
__init__.py  
intent_agent.py  
discovery_agent.py  
ranking_agent.py  
pricing_agent.py  
booking_agent.py  
followup_agent.py  
prompts/  
intent.md (empty placeholder; Product owns this)  
ranking.md  
pricing.md  
booking.md  
followup.md  
data/  
providers.json (empty array [] for now)  
intent_examples.jsonl (empty file)  
tools/  
__init__.py  
tests/  
__init__.py  
test_health.py  
test_orchestrate.py  
scripts/  
eval_intent.py (stub for now)  
  
PYTHON: 3.11
```

DEPENDENCIES (in pyproject.toml):

- python = "^3.11"
- fastapi = "^0.110"
- "uvicorn[standard]" = "^0.27"
- google-adk = "^0.5"
- google-genai = "^0.3"
- pydantic = "^2.6"
- pydantic-settings = "^2.2"
- google-cloud-firestore = "^2.15"
- google-cloud-storage = "^2.14"
- reportlab = "^4.0"
- httpx = "^0.27"
- structlog = "^24.1"
- (dev) pytest = "^8.0"
- (dev) pytest-asyncio = "^0.23"
- (dev) ruff = "^0.4"

PYDANTIC MODELS (in app/models.py) — define ALL of these exactly:

```
class Intent(BaseModel):
    service_type: Literal["plumber", "electrician", "ac_technician", "geyser_technician", "carpenter", "painter", "beautician", "tutor", "appliance_repair", "gas_leak_specialist"]
    location: Optional[str] = None
    city: str = "Islamabad"
    time_window: str
    urgency: Literal["emergency", "high", "normal", "flexible"]
    job_complexity: Literal["basic", "intermediate", "complex"]
    specialization_hint: Optional[str] = None
    gender_preference: Literal["male", "female", "any"] = "any"
    budget_range: Optional[Tuple[int, int]] = None
    raw_notes: Optional[str] = None
    confidence: float # 0.0 to 1.0
    clarification_question: Optional[str] = None
```

```

class ProviderCandidate(BaseModel):
    id: str
    name: str
    service_categories: List[str]
    specializations: List[str] = []
    distance_km: float
    neighborhood: str
    rating: float
    completed_jobs_in_area: int
    on_time_score: float
    cancellation_rate: float
    review_recency_days: int
    risk_score: float
    current_workload: float
    availability_status: Literal["available_now", "next_slot_within_2h", "next_slot_today", "tomorrow_or_later"]
    next_slot: str # ISO datetime
    base_visit_fee_pkr: int
    rate_per_hour_pkr: int
    gender: Literal["male", "female"]
    years_experience: int
    languages: List[str] = []
    verified: bool = True
    phone_masked: str
    alternate_slot_reason: Optional[str] = None # only on alternates

class DiscoveryResult(BaseModel):
    candidates: List[ProviderCandidate]
    alternates: List[ProviderCandidate] = []
    no_match_reason: Optional[str] = None

class FactorScores(BaseModel):
    distance: float
    availability: float
    rating: float
    on_time: float
    specialization: float
    risk: float
    total: float

class RankingResult(BaseModel):
    recommended_id: Optional[str]
    top_three_ids: List[str] = []
    factor_scores: Dict[str, FactorScores] = {}
    reasoning_english: str = ""
    reasoning_urdu: str = ""
    tradeoffs: str = ""
    confidence: Literal["high", "medium", "low"] = "medium"
    error: Optional[str] = None

```

```

class PriceLineItem(BaseModel):
    label_english: str
    label_urdu: str
    amount_pkr: int # negative for discounts
    kind: Literal["base","addon","multiplier","discount","surge"]

class PriceQuote(BaseModel):
    quote_id: str
    line_items: List[PriceLineItem]
    subtotal_pkr: int
    estimated_total_pkr: int
    estimated_range_pkr: Tuple[int,int]
    explanation_english: str
    explanation_urdu: str
    fairness_note: str
    expires_at: str # ISO

class BookingLifecycle(BaseModel):
    confirmed_at: Optional[str] = None
    en_route_at: Optional[str] = None
    arrived_at: Optional[str] = None
    in_progress_at: Optional[str] = None
    completed_at: Optional[str] = None
    cancelled_at: Optional[str] = None

class Booking(BaseModel):
    booking_id: str
    user_id: str
    provider_id: str
    service_type: str
    location: str
    city: str
    scheduled_time: str
    status: Literal["confirmed","en_route","arrived","in_progress","completed","cancelled","disputed"] =
    "confirmed"
    lifecycle: BookingLifecycle
    accepted_quote: PriceQuote
    intent_snapshot: Intent
    ranking_snapshot: Optional[RankingResult] = None
    receipt_url: Optional[str] = None
    confirmation_message_english: str = ""
    confirmation_message_urdu: str = ""

class FollowupResult(BaseModel):
    mode: Literal["reminder","checkin","dispute"]
    english: str = ""
    urdu: str = ""
    cta: Literal["rate","contact_provider","none"] = "none"
    classification: Optional[str] = None
    resolution: Optional[str] = None
    refund_pkr: int = 0
    escalate_to_human: bool = False

```

```

class OrchestrateRequest(BaseModel):
    text: str
    user_id: str
    user_location: Optional[Tuple[float,float]] = None # (lat, lng)

class OrchestrateResponse(BaseModel):
    intent: Intent
    discovery: DiscoveryResult
    ranking: RankingResult
    pricing: PriceQuote
    booking_preview: Dict # booking-shaped; not committed until /book
    followup_planned: Dict # reminder timing only

CONFIG (app/config.py) using pydantic-settings:
class Settings(BaseSettings):
    GEMINI_API_KEY: str
    GOOGLE_CLOUD_PROJECT: str = "bulao-hackathon"
    DEMO_MODE: bool = False
    FIRESTORE_EMULATOR_HOST: Optional[str] = None
    LOG_LEVEL: str = "INFO"

class Config:
    env_file = ".env"
    env_file_encoding = "utf-8"

settings = Settings()

FASTAPI APP (app/main.py):
- Use lifespan context to init logger + ADK client at startup
- CORS middleware allowing all origins (hackathon scope)
- Global exception handler returning JSON {error: str, code: int}
- Three endpoints:
GET /health -> {status: "ok", version: "0.1.0", commit: }
POST /orchestrate -> calls orchestrator.run_pipeline(req); returns OrchestrateResponse
POST /book -> stub for Day 3; returns 501 Not Implemented for now

ORCHESTRATOR (app/orchestrator.py):
- async def run_pipeline(req: OrchestrateRequest) -> OrchestrateResponse
- For Day 1, returns hardcoded mock JSON matching the schemas (use realistic Pakistani names).
- Specifically: intent for "plumber chahiye" -> service_type=plumber, location=null, urgency=high,
confidence=0.78
- Use one consistent mock provider "Ali Plumbing Works" in G-13

AGENT PLACEHOLDERS (app/agents/*.py):
- Each file exports: async def run(input_data: ) ->
- Day 1 implementation: just return hardcoded mock data matching the schema
- Add a TODO comment: "# Day N: Replace with real Gemini call. See app/prompts/.md."

LOGGING (set up in app/main.py lifespan):
- structlog with JSON renderer to stdout
- Bind {service: "bulao-backend", env: }
- Each agent's run() logs: {"event": "agent_start|"agent_end", "agent": , "duration_ms": N}

```

DOCKERFILE (multi-stage):

```
# Stage 1: build
FROM python:3.11-slim AS builder
RUN pip install poetry==1.7.1
COPY pyproject.toml poetry.lock* ./
RUN poetry config virtualenvs.in-project true && poetry install --no-root --without dev
```

```
# Stage 2: runtime
FROM python:3.11-slim
WORKDIR /app
COPY --from=builder .venv ./venv
COPY app ./app
ENV PATH="/app/.venv/bin:$PATH" PORT=8080
EXPOSE 8080
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8080"]
```

.env.example:

```
GEMINI_API_KEY=paste-your-key-here
GOOGLE_CLOUD_PROJECT=bulao-hackathon
DEMO_MODE=false
LOG_LEVEL=INFO
```

.gitignore: standard Python (.venv, __pycache__, *.pyc) + .env + dist/ + .pytest_cache + .ruff_cache

README.md: 4-section skeleton — Overview, Run locally, Deploy, Architecture (one paragraph each)

TESTS (app/tests/):

- test_health.py: assert GET /health returns 200 with {"status":"ok"}
- test_orchestrate.py: POST /orchestrate with {text:"plumber chahiye", user_id:"u_test"} returns 200 with all 6 keys present, intent.service_type == "plumber", confidence between 0 and 1

VERIFY (run these and confirm they all pass):

1. cd backend && poetry install
2. poetry run uvicorn app.main:app --reload --port 8080 &
3. curl http://localhost:8080/health -> {"status":"ok","version":"0.1.0"}
4. curl -X POST http://localhost:8080/orchestrate -H 'Content-Type: application/json' -d '{"text":"plumber chahiye","user_id":"u_test"}' -> JSON with intent, discovery, ranking, pricing, booking_preview, followup_planned
5. poetry run pytest -> 2 passed
6. docker build -t bulao-backend . -> succeeds
7. docker run --rm -p 8080:8080 -e GEMINI_API_KEY=fake bulao-backend & sleep 5 && curl http://localhost:8080/health -> {"status":"ok"}

OUTPUT WHEN DONE:

- All files created exactly as specified.
- Plan Artifact shown to the developer for review BEFORE any code is written.
- Final tool call summary lists every file created with its byte count.

If you need to deviate from this spec for any reason, STOP and ask the developer in the Plan Artifact. Do not assume.

End-of-day verification

- poetry run pytest shows 2 passed.
 - curl localhost:8080/health returns {"status":"ok","version":"0.1.0"}.
 - curl -X POST localhost:8080/orchestrate with a sample body returns all 6 agent keys.
 - docker build . succeeds.
 - Backend repo committed with the full file tree; Mobile teammate can git pull.
 - Infra teammate has the URL pattern needed to deploy this image.
 - Screenshot of Antigravity's Plan Artifact saved to docs/antigravity-trace/day1/.
- *Send Mobile and Infra the /orchestrate stub response JSON as a GitHub Gist comment or WhatsApp message. They need the exact field names to mock against.*

05 · DAY 3 · MAY 16

Day 2 · orchestrator + Firestore setup

Build the real FastAPI orchestrator and set up Firestore. By end of day, Mobile gets real Intent + Discovery data through /orchestrate.

Day 2 · stub orchestrator + Firestore setup

Backend Agents scaffold is live since yesterday evening. Today you wire together the endpoints and set up Firestore so there is a real database ready when Booking arrives tomorrow. By end of day, Mobile gets a working /orchestrate that returns real Intent + Discovery data with stub Ranking, Pricing, Booking, and Follow-up.

PROMPT · BUILD THE ORCHESTRATOR + FIRESTORE SETUP

Build the FastAPI application and wire it to the Backend Agents teammate's agent modules.

FILES TO CREATE/MODIFY:

1. app/main.py — the FastAPI application entry point:

```
from fastapi import FastAPI
from app.routers import orchestrate, book, dispute, health, rating
app = FastAPI(title="Bulao Backend", version="1.0.0")
app.include_router(orchestrate.router)
app.include_router(book.router)
app.include_router(dispute.router)
app.include_router(health.router)
app.include_router(rating.router)
```

2. app/routers/orchestrate.py:

```
POST /orchestrate {text: str, user_id: str} -> OrchestrateResponse
For today (Day 2): call real intent_agent.run() + discovery_agent.run().
For Ranking/Pricing/Booking/Followup: call the stubs (app.agents.stubs).
Return the full OrchestrateResponse shape even if stubs are placeholder.
```

3. app/routers/book.py:

```
POST /book {quote_id: str, user_id: str} -> BookingRecord
For today: stub only — return a BookingRecord with status="confirmed" and
a fake booking_id generated as: f"BUL-{date.today().strftime('%Y%m%d')}-{uuid4().hex[:6].upper()}"
```

4. app/routers/dispute.py:

```
POST /dispute {booking_id, rating, complaint, checklist, user_id} -> DisputeResult
For today: stub only — return DisputeResult(classification="quality_complaint",
resolution="partial_refund", refund_pkr=1000, message_urdu="...", message_english="...",
escalate_to_human=False)
```

5. app/routers/health.py:

```
GET /health -> {"status": "ok", "timestamp": }
```

6. app/routers/rating.py:

```
POST /rating {booking_id, rating, user_id} -> {"status": "ok", "message_urdu": "Shukriya!"}
```


FIRESTORE SETUP:

Create app/db/firestore_client.py:

```
from google.cloud import firestore
db = firestore.AsyncClient(project="bulao-hackathon")
async def save_booking(booking: dict) -> None:
    await db.collection("bookings").document(booking["booking_id"]).set(booking)
async def get_booking(booking_id: str) -> dict | None:
    doc = await db.collection("bookings").document(booking_id).get()
    return doc.to_dict() if doc.exists else None
```

For Day 2, the endpoints DON'T call Firestore yet — just log the booking_id.

Add a TODO comment: # TODO Day 3: call firestore_client.save_booking(booking)

After wiring:

cd backend

poetry run uvicorn app.main:app --reload

curl -X POST http://localhost:8000/orchestrate -H "Content-Type: application/json" -d '{"text": "Mujhe G-13 mein kal subah AC theek karwana hai", "user_id": "test_user"}'

Confirm the response has intent, discovery, ranking (stub), pricing (stub), booking_preview.

Commit: git commit -am "feat(workflows): orchestrator + all endpoint stubs + Firestore client"

git push

POST IN WHATSAPP: "Workflows: /orchestrate is live. Real Intent + Discovery. Stubs for the rest."

Late afternoon · lifecycle GET endpoint

Mobile polls /booking/{id}/lifecycle every 10 seconds. Build it today as a stub that returns a realistic rotating status so Mobile can build the LifecycleScreen without needing real data.

PROMPT · BUILD GET /BOOKING/{ID}/LIFECYCLE

Add the lifecycle polling endpoint at app/routers/lifecycle.py.

Route: GET /booking/{booking_id}/lifecycle

Response:

```
{
  "booking_id": "",
  "status": "",
  "eta_minutes": ,
  "lifecycle": {
    "confirmed_at": "",
    "en_route_at": "",
    "arrived_at": "",
    "in_progress_at": "",
    "completed_at": ""
  }
}
```

For Day 2 (stub): return a rotating status based on booking creation time:

0-30s after booking: confirmed
30-90s: en_route (eta_minutes=12)
90-150s: arrived (eta_minutes=null)
150-300s: in_progress
>300s: completed

This allows Mobile to test all lifecycle states without a real provider simulation.

Register the router in app/main.py:

```
from app.routers import lifecycle
app.include_router(lifecycle.router)
```

Test:

```
curl http://localhost:8000/booking/BUL-20260516-9KP3X4/lifecycle
```

Commit: git commit -am "feat(lifecycle): GET /booking/{id}/lifecycle stub"

End-of-day verification

- GET /health returns 200 with status=ok.
- POST /orchestrate returns valid JSON with all 6 agent output keys.
- Intent + Discovery show REAL data; Ranking/Pricing/Booking/Followup show stubs.
- POST /book returns a BookingRecord with a valid booking_id.
- GET /booking/{id}/lifecycle returns a rotating status (stub simulation).
- POST /dispute returns a stub DisputeResult.
- Firestore client is written but not yet called (TODO comment in place).
- All endpoints tested with curl or Insomnia; no 5xx responses.

06 · DAY 4 · MAY 17

Day 3 · Booking Agent + wire full pipeline

Booking Agent goes live. At noon, Backend Agents hands off Ranking + Pricing — wire them into /orchestrate. By 6pm, the full 5-agent pipeline (Intent → Discovery → Ranking → Pricing → Booking) returns real data end-to-end.

Morning · the Booking Agent

Booking finalizes the transaction: writes to Firestore, generates the receipt PDF, and drafts the WhatsApp-style confirmation message using the Booking Agent LLM call. The receipt is the demo's hero asset — design it.

PROMPT · DAY 3 — BOOKING AGENT

ROLE: Build the Booking Agent. It takes the accepted PriceQuote, writes a booking record to Firestore (or local JSON fallback), generates a receipt PDF, drafts a WhatsApp-style confirmation message in Urdu + English, and initializes the service-quality lifecycle.

FILES TO EDIT/CREATE:

- app/agents/booking_agent.py (replace stub)
- app/prompts/booking.md (paste from Product team)
- app/tools/pdf_receipt.py (NEW — generates the receipt PDF)
- app/tests/test_booking_agent.py (NEW)
- New endpoint POST /book in app/main.py

POST /book endpoint (in app/main.py):

- Request body: {quote_id: str, user_id: str, user_name: Optional[str], intent: Intent, provider_id: str, accepted_quote: PriceQuote}
- Calls booking_agent.run(...) -> returns Booking
- Returns 200 with Booking (including receipt_url)

IMPLEMENTATION (app/agents/booking_agent.py):

```
import json, time, uuid, asyncio
from datetime import datetime
from pathlib import Path
from app.models import Intent, ProviderCandidate, PriceQuote, Booking, BookingLifecycle

async def run(intent: Intent, provider: ProviderCandidate, accepted_quote: PriceQuote,
user_id: str, user_name: str = None) -> Booking:
    log.info("agent_start", agent="booking")
    t0 = time.monotonic()

    booking_id = f"BUL-{datetime.now().strftime('%Y%m%d')}-{uuid.uuid4().hex[:6].upper()}"
    now_iso = datetime.now().isoformat() + "+05:00"
```

```

# Generate receipt PDF
from app.tools.pdf_receipt import generate_receipt
receipt_path = generate_receipt(booking_id, intent, provider, accepted_quote, user_name)
# Upload to Cloud Storage (if FIRESTORE_EMULATOR not set; else local file URL)
receipt_url = _upload_or_local(receipt_path, booking_id)

# Draft confirmation messages via LLM
payload = {
    "booking": {"booking_id": booking_id, "service_type": intent.service_type,
    "scheduled_time": _interpret_time(intent.time_window),
    "location": intent.location, "accepted_quote": accepted_quote.model_dump()},
    "provider": {"name": provider.name, "rating": provider.rating,
    "years_experience": provider.years_experience},
    "user_name": user_name,
}
try:
    raw = await _llm_agent.invoke(json.dumps(payload, ensure_ascii=False))
    msgs = json.loads(raw)
    msg_en = msgs.get("english", "")
    msg_ur = msgs.get("urdu", "")
except Exception as e:
    log.error("booking_llm_failure", error=str(e))
    msg_en = f"Booking confirmed. ID: {booking_id}. {provider.name} will reach {intent.location}. — Bulao"
    msg_ur = f"Booking confirm ho gaya. ID: {booking_id}. {provider.name} {intent.location} pohanch jayenge. — Bulao"

booking = Booking(
    booking_id=booking_id, user_id=user_id, provider_id=provider.id,
    service_type=intent.service_type, location=intent.location or "",
    city=intent.city, scheduled_time=_interpret_time(intent.time_window),
    status="confirmed",
    lifecycle=BookingLifecycle(confirmed_at=now_iso),
    accepted_quote=accepted_quote, intent_snapshot=intent,
    receipt_url=receipt_url,
    confirmation_message_english=msg_en, confirmation_message_urdu=msg_ur,
)

# Persist
_persist(booking)

log.info("agent_end", agent="booking", duration_ms=int((time.monotonic()-t0)*1000),
    booking_id=booking_id)
return booking

PERSIST: try Firestore; fall back to local JSON file at app/data/bookings_local.json
RECEIPT PDF (app/tools/pdf_receipt.py): use reportlab; A5 size; Bulao header with gold accent; booking ID;
provider line; scheduled time; line items table; total; fine print "Service may take 1-3 hours."

TESTS:
- a) booking_id matches format BUL-YYYYMMDD-XXXXXX
- b) Receipt PDF file is created and is > 1KB
- c) Confirmation message contains booking_id, provider name, and a time reference
- d) Urdu message contains at least one Urdu marker (hai, ke, ka, ki, mein)
- e) Booking.lifecycle.confirmed_at is populated and parses as ISO 8601

```

VERIFY:

1. poetry run pytest app/tests/test_booking_agent.py -> 5 pass
2. POST /book with a valid quote -> returns Booking with receipt_url; open the URL in a browser, the PDF looks designed (not raw text dump)

Afternoon · wire real Ranking + Pricing into orchestrator

Backend Agents teammate handed off real Ranking and Pricing modules at noon. Replace the stubs in /orchestrate with the real calls. End-to-end pipeline now runs all four intelligence agents + Booking.

PROMPT · WIRE RANKING + PRICING + BOOKING INTO /ORCHESTRATE

Update app/routers/orchestrate.py to call the real Ranking, Pricing, and Booking agents.

CONTEXT: As of today noon, these modules are available from the Backend Agents teammate's commit:

app.agents.ranking_agent — run(intent, candidates) -> RankingResult

app.agents.pricing_agent — run(intent, provider, market_demand, is_first_booking) -> PriceQuote

STEPS:

1. In app/routers/orchestrate.py, replace the stub calls for ranking and pricing:

```
ranking = await ranking_agent.run(intent=intent_result, candidates=discovery_result.candidates)
pricing = await pricing_agent.run(
    intent=intent_result,
    provider=next(c for c in discovery_result.candidates if c.id == ranking.recommended_id),
    market_demand=0.5, # start with 0.5; real market_demand later
    is_first_booking=True # simplification for demo
)
```
 2. After pricing, call booking_agent.preview(intent, provider, pricing) to get the BookingPreview (NOT the full booking — that only happens after /book is called).
 3. Return the full OrchestrateResponse with real ranking + pricing + booking_preview.
 4. In app/routers/book.py, replace the stub:
 After the user confirms (POST /book with a quote_id):
 - Verify the quote_id matches a cached PriceQuote (store in a Redis or in-process dict keyed by quote_id)
 - Check that it has not expired (pricing.expires_at > now())
 - Call booking_agent.run(booking, provider, user_name) to get the confirmation message
 - Save the booking to Firestore: await firestore_client.save_booking(booking.model_dump())
 - Return BookingRecord
 5. Handle quote_expired (400):
 If the quote is older than 15 minutes: return 400 with {"error": "quote_expired"}.
 Mobile shows a retry UX in this case.
- Test end-to-end:
 curl -X POST http://localhost:8000/orchestrate -d '{"text": "Mujhe G-13 mein kal subah AC theek karwana hai", "user_id": "test"}'
- Verify response includes real ranking.factor_scores and pricing.line_items.
- Commit: git commit -am "feat(orchestrator): wire real Ranking + Pricing + Booking"

End-of-day verification

- Ranking Agent factor_scores total to a sensible weighted sum; reasoning has 3+ digits.
- Pricing Agent produces 4-7 line_items in correct order; range is always ≥ 250 PKR wide.
- Booking returns a Booking object with a receipt_url that opens to a designed PDF.
- All three new agents are wired into /orchestrate; end-to-end response time < 8 seconds.
- Three Antigravity Plan Artifacts saved: one per new agent.
- Mobile teammate has confirmed the response shape matches their UI expectations.

07 · DAY 5 · MAY 18

Day 4 · Follow-up & Dispute + Cloud Scheduler

Sixth agent. Dispute resolution. Cloud Scheduler integration. By end of day, the full six-agent pipeline runs, the dispute path is demonstrable, and the DEMO_MODE fallback means the demo can run without live Scheduler.

Morning · Follow-up & Dispute Agent

The sixth and final agent. Two modes: reminder/check-in on the happy path, and dispute classification + resolution when something goes wrong. Cloud Scheduler fires the reminder 30 minutes before the service.

PROMPT · DAY 4 — FOLLOW-UP & DISPUTE AGENT

ROLE: Build the Follow-up & Dispute Agent. Two modes: (1) reminder/check-in for the happy path, (2) dispute classification + resolution when rating < 3 or user reports an issue.

FILES TO EDIT/CREATE:

- app/agents/followup_agent.py (replace stub)
- app/prompts/followup.md (paste from Product team)
- app/main.py: add POST /dispute and POST /lifecycle endpoints
- app/tests/test_followup_agent.py (NEW)
- scripts/trigger_reminders.py (NEW — Cloud Scheduler will call this hourly)

NEW ENDPOINTS:

POST /lifecycle (mobile calls this to advance booking through states)
 Body: {booking_id: str, new_status: "en_route"|"arrived"|"in_progress"|"completed"}
 Updates Firestore: bookings/{booking_id}.lifecycle._at = now
 Returns: updated Booking

POST /dispute (mobile calls this when user reports an issue OR rates < 3)

Body: {
 booking_id: str,
 user_complaint: str, # free text in Urdu/English
 service_checklist: List[{item: str, status: "yes"|"no"}],
 rating: int # 1-5
 }

Calls followup_agent.run(mode="dispute", ...) and returns FollowupResult with classification + resolution + refund_pkr + message.

FOLLOWUP AGENT IMPLEMENTATION:

```
async def run(mode: Literal["reminder", "checkin", "dispute"], booking: Booking,
provider: ProviderCandidate, user_complaint: str = None,
service_checklist: List[Dict] = None, rating: int = None) -> FollowupResult:
log.info("agent_start", agent="followup", mode=mode, booking_id=booking.booking_id)
t0 = time.monotonic()
```

```

if mode in ("reminder", "checkin"):
    return await _run_reminder_or_checkin(mode, booking, provider)
elif mode == "dispute":
    return await _run_dispute(booking, provider, user_complaint, service_checklist, rating)

async def _run_reminder_or_checkin(mode, booking, provider):
    payload = {"mode": mode, "booking": booking.model_dump(), "provider": provider.model_dump()}
    raw = await _llm_agent.invoke(json.dumps(payload, ensure_ascii=False))
    data = json.loads(raw)
    return FollowupResult(
        mode=mode,
        english=data.get("english", ""),
        urdu=data.get("urdu", ""),
        cta=data.get("cta", "none"),
    )

async def _run_dispute(booking, provider, user_complaint, service_checklist, rating):
    # Deterministic pre-classification based on rules
    auto_classification = None
    auto_resolution = None
    auto_refund = 0

    if rating == 1:
        auto_classification = "quality_complaint"
        auto_resolution = "full_refund"
        auto_refund = booking.accepted_quote.estimated_total_pkr
    elif rating == 2:
        auto_classification = "quality_complaint"
        auto_resolution = "partial_refund"
        auto_refund = int(booking.accepted_quote.estimated_total_pkr * 0.5)

    # Look at checklist: if more than 50% are "no", it's a no_show or quality_complaint
    if service_checklist:
        no_count = sum(1 for item in service_checklist if item["status"] == "no")
        if no_count >= len(service_checklist) * 0.5:
            auto_classification = auto_classification or "quality_complaint"

```



```

# Pass to LLM for final classification + message generation, with our suggestion as hint
payload = {
    "mode": "dispute",
    "booking": booking.model_dump(),
    "provider": provider.model_dump(),
    "user_complaint": user_complaint,
    "service_checklist": service_checklist or [],
    "rating": rating,
    "auto_suggestion": {
        "classification": auto_classification,
        "resolution": auto_resolution,
        "refund_pkr": auto_refund,
    },
}
try:
    raw = await _llm_agent.invoke(json.dumps(payload, ensure_ascii=False))
    data = json.loads(raw)
except Exception as e:
    log.error("dispute_llm_failure", error=str(e))
# Fallback to deterministic resolution
return FollowupResult(
    mode="dispute",
    classification=auto_classification or "other",
    resolution=auto_resolution or "escalate_to_human",
    refund_pkr=auto_refund,
    english=f"We're sorry for the issue. We're processing a refund of PKR {auto_refund:}. — Bulao",
    urdu=f"Maafi mangte hain. PKR {auto_refund:} ka refund process kar rahe hain. — Bulao",
    escalate_to_human=(auto_resolution is None),
)

return FollowupResult(
    mode="dispute",
    classification=data.get("classification", auto_classification or "other"),
    resolution=data.get("resolution", auto_resolution or "escalate_to_human"),
    refund_pkr=data.get("refund_pkr", auto_refund),
    english=data.get("message_english", ""),
    urdu=data.get("message_urdu", ""),
    escalate_to_human=data.get("escalate_to_human", False),
)

PERSIST DISPUTES: bookings/{booking_id}/dispute subcollection with {raised_at, user_complaint,
service_checklist, rating, classification, resolution, refund_pkr, resolved_at}

DEMO_MODE FALLBACK:
- If settings.DEMO_MODE is true, /orchestrate returns a pre-canned response loaded from
app/data/demo_canned_response.json
- Generate that file now with a complete demo flow: intent ("Mujhe G-13 mein kal subah AC theek karwana
hai"), 4 candidates, ranked recommendation with Urdu reasoning, full pricing breakdown with 6 line items,
booking preview
- Loud log at startup if DEMO_MODE=true: "[!!!] DEMO_MODE IS ACTIVE — NOT FOR PRODUCTION"

```

TESTS:

- a) Rating 1 -> classification="quality_complaint", resolution="full_refund"
- b) Rating 2 -> partial_refund with 50% of total
- c) Checklist with >=50% "no" responses triggers quality_complaint classification
- d) DEMO_MODE returns canned response regardless of input
- e) Reminder mode returns urdu and english non-empty
- f) Dispute mode includes a message that acknowledges feeling before stating resolution

VERIFY:

1. poetry run pytest app/tests/test_followup_agent.py -> 6 pass
2. POST /dispute with rating=1 -> classification="quality_complaint", resolution="full_refund"
3. POST /lifecycle progressing through all 4 states works
4. DEMO_MODE=true uvicorn ... -> all responses come from the canned file, ~50ms response time

Late afternoon · Cloud Scheduler integration

The Follow-up Agent fires on a schedule, not on demand. Set up Cloud Scheduler to POST to /followup/trigger every 5 minutes; the endpoint checks which bookings need a reminder or check-in and fires the agent for those.

PROMPT · WIRE CLOUD SCHEDULER FOR FOLLOW-UP TRIGGERS

Add the Cloud Scheduler trigger mechanism to the Follow-up Agent.

CREATE app/routers/followup_trigger.py:

Route: POST /followup/trigger

This route is called by Cloud Scheduler (not by the mobile app).

Authentication: verify the request has the Cloud Scheduler service account header.

from google.auth.transport import requests as google_requests

from google.oauth2 import id_token

Verify the OIDC token in the Authorization header

LOGIC:

1. Query Firestore for all bookings with status="confirmed" where:
 - a) scheduled_time is within the next 35 minutes (reminder window) AND lifecycle.en_route_at is null
 - b) OR completed_at is between 2h ago and 2.5h ago AND followup_sent is not True (check-in window)
2. For each matching booking, call followup_agent.run(booking, provider, type="reminder"|"checkin")
3. Log the sent messages (don't send real SMS/WhatsApp for the demo — just log them)
4. Update Firestore: set reminder_sent=True or checkin_sent=True on the booking

For the DEMO (since we don't have real Cloud Scheduler in the demo):

Add a DEMO_MODE check: if os.environ.get("DEMO_MODE") == "true", the

/orchestrate endpoint automatically "fires" a reminder after the booking

is created by calling followup_agent.run() directly with type="reminder".

This is the demo safety net — no Scheduler needed on the demo phone.

REGISTER in app/main.py:

from app.routers import followup_trigger

app.include_router(followup_trigger.router)

CLOUD SCHEDULER SETUP (Infra teammate will create the scheduler, but document the job spec):

Create docs/cloud-scheduler-job.json:

```
{
  "name": "bulao-followup-trigger",
  "schedule": "*/*5 * * * *",
  "timeZone": "Asia/Karachi",
  "httpTarget": {
    "uri": "https://followup/trigger",
    "httpMethod": "POST",
    "oidcToken": {"serviceAccountEmail": "bulao-scheduler@bulao-hackathon.iam.gserviceaccount.com"}
  }
}
```

Commit: git commit -am "feat(followup): Cloud Scheduler trigger endpoint + DEMO_MODE fallback"

End-of-day verification

- All 6 agents are wired in the orchestrator with real Gemini calls.
- POST /dispute with rating=1 returns full_refund classification.
- POST /lifecycle transitions through confirmed → en_route → arrived → in_progress → completed.
- DEMO_MODE=true serves a complete canned response in <100ms.
- All 7 edge case tests pass.
- Mobile teammate confirms dispute UI works end-to-end against the real backend.
- Antigravity Plan Artifact for dispute flow saved to docs/antigravity-trace/day4/.

08 · DAY 6 · MAY 19

Day 5 · polish, Docker, handoff to Infra

Final hardening. Docker image tagged v1.0.0. deploy.sh tested. Infra teammate gets everything they need to submit.

Full day · final polish + Docker

No new features. Tests, Docker, handoff.

PROMPT · DAY 5 — POLISH AND HANDOFF

ROLE: Final polish day. No new agents. Goal: every code path is robust, all tests pass, the production Docker image is tagged v1.0.0 and pushed to Artifact Registry, and the Infra teammate has everything they need to ship.

TASKS (in order, no LLM needed except for content review):

1. Run the full test suite. Every test must pass.

```
poetry run pytest -v
```

If any fail, fix them. Do not skip.

2. Run ruff and fix all linting issues.

```
poetry run ruff check . --fix
```

```
poetry run ruff format .
```

3. Run the eval one more time.

```
poetry run python scripts/eval_intent.py
```

Confirm: service_type accuracy >= 92%, job_complexity accuracy >= 85%.

4. End-to-end smoke test against deployed Cloud Run (Infra owns the URL).

```
curl https:///health -> 200
```

Run through 5 different real Urdu sentences from the dataset, verify every response is sensible.

5. Update version. In app/main.py change version to "1.0.0". In Dockerfile add LABEL version="1.0.0".

6. Build and tag the production image.

```
docker build -t asia-south1-docker.pkg.dev/bulao-hackathon/bulao/backend:v1.0.0 -t
```

```
asia-south1-docker.pkg.dev/bulao-hackathon/bulao/backend:latest .
```

```
docker push asia-south1-docker.pkg.dev/bulao-hackathon/bulao/backend:v1.0.0
```

```
docker push asia-south1-docker.pkg.dev/bulao-hackathon/bulao/backend:latest
```

7. Update README.md (final version).

Sections (each 1-2 paragraphs):

- Overview (what Bulao is)
- Architecture (link to /docs/diagrams/architecture.png)
- Six agents (one paragraph each)
- Setup & run locally
- Deploy to Cloud Run
- Tests & eval
- Environment variables
- Data files & schemas
- Known limitations
- License (MIT)

8. Generate the final canned demo response.

Use a real /orchestrate call: 'Mujhe G-13 mein kal subah AC theek karwana hai'

Capture the response, save to app/data/demo_canned_response.json.

Verify by running DEMO_MODE=true ... and confirming the response matches.

9. Final handoff package for Infra (push these to the repo):

- The production image tag (v1.0.0)
- The Firestore export JSON (gcloud firestore export -> share GCS URI)
- A short HANDOFF.md listing: image URI, deployed Cloud Run URL, env vars required, who owns what after submission

10. Final Antigravity screenshot capture (for the Trace deliverable):

- One Manager View screenshot showing the full repo
- One Plan Artifact screenshot for a representative agent
- The test suite output screenshot showing all green

Save to docs/antigravity-trace/day5/.

VERIFY:

1. poetry run pytest -v -> all green
2. ruff check . -> no issues
3. docker images | grep bulao-hackathon/bulao/backend:v1.0.0 -> exists locally AND in Artifact Registry
4. curl https://health on the v1.0.0 image -> {"status":"ok","version":"1.0.0"}
5. README is complete; HANDOFF.md is committed; Infra teammate has read it.

OUTPUT: Antigravity reports back with the test summary, the image digest, and a one-line confirmation that the handoff is complete.

End-of-day verification

- All tests pass (poetry run pytest -v).
- Linter clean (ruff check .).
- Intent eval $\geq 92\%$ service_type, $\geq 85\%$ complexity.
- Image v1.0.0 pushed to Artifact Registry.
- Cloud Run /health returns version: "1.0.0".
- README.md and HANDOFF.md committed.

- Final canned demo response saved and verified.
- Day 5 screenshots in docs/antigravity-trace/day5/.

APPENDIX A

All five HTTP endpoints.

The exact URL paths, methods, request/response shapes, and error codes that Mobile depends on. Frozen from Day 1 evening.

```
/orchestrate POST – Full pipeline (all 6 agents in sequence)
/book POST      – Accept the PriceQuote, write Firestore, generate receipt
/booking/{id}/lifecycle GET – Poll lifecycle status (10s interval from Mobile)
/dispute POST   – Submit rating < 3 + complaint + checklist
/rating POST    – Submit rating >= 3 (no dispute path)
/followup/trigger POST – Cloud Scheduler callback (not called by mobile)
/health GET     – {"status": "ok", "timestamp": "<ISO>"}
```

All routes return application/json.

All error responses: {"error": "<code>", "message": "<human-readable>"}

Error codes: quote_expired, provider_not_found, booking_not_found, invalid_input

APPENDIX B

The full /orchestrate response JSON.

The exact response shape Mobile parses. Every field name is fixed. Null fields for stubs are fine; missing fields are not.

```
// The API the Mobile teammate is wiring to.
// All endpoints accept and return JSON. Errors come as {error: str, code: int} with HTTP 4xx/5xx.

GET /health
-> 200 {"status":"ok", "version":"<semver>", "commit":"<git-sha>"}

POST /orchestrate
Body: {
  text: str,           // user input – Urdu / Roman Urdu / English / mixed
  user_id: str,        // stable per-user identifier from the mobile app
  user_location: [lat: float, lng: float] // optional
}
-> 200 {
  intent: Intent,
  discovery: DiscoveryResult,
  ranking: RankingResult,
  pricing: PriceQuote,
  booking_preview: {    // not yet committed; user must call /book to commit
    provider_id: str,
    scheduled_time: str,
    expected_total_pkr: int
  },
  followup_planned: {
    reminder_at: str,    // ISO datetime
    checkin_at: str
  },
  warnings: []           // list of degraded-mode warnings, e.g. "intent_fallback_keyword"
}
-> 200 with needs_clarification=true if intent.confidence < 0.7:
  { intent, needs_clarification: true, clarification_question: str }
```



```
-> 200 with no_match_reason if discovery returns empty:
  { intent, discovery: {candidates:[], alternates:[], no_match_reason: str},
    user_message_urdu: str, user_message_english: str }

POST /book
Body: {
  quote_id: str,
  user_id: str,
  user_name: str | null,
  intent: Intent,
  provider_id: str,
  accepted_quote: PriceQuote
}
-> 200 Booking
-> 409 {"error":"quote_expired"} | {"error":"slot_taken"}

POST /lifecycle
Body: {booking_id: str, new_status: "en_route"|"arrived"|"in_progress"|"completed"}
-> 200 Booking (updated)

POST /dispute
Body: {
  booking_id: str,
  user_complaint: str,
  service_checklist: [{item: str, status: "yes"|"no"}],
  rating: int    // 1-5
}
-> 200 FollowupResult (mode="dispute") with classification, resolution, refund_pkr, urdu/english
messages
```

APPENDIX C

Firestore bookings collection schema.

The exact document structure stored in Firestore. Every field that the Mobile app or Cloud Scheduler reads must be present at write time.

```
{
  // Collection: bookings
  // Document ID: booking_id (e.g. "BUL-20260516-9KP3X4")

  "booking_id": "BUL-20260516-9KP3X4",
  "user_id": "usr_abc123",
  "provider_id": "prov_042",
  "service_type": "ac_technician",
  "location": "G-13",
  "city": "Islamabad",
  "scheduled_time": "2026-05-17T10:00:00+05:00",
  "urgency": "normal",
  "job_complexity": "intermediate",
  "status": "confirmed",

  // Lifecycle tracking
  "lifecycle": {
    "confirmed_at": "2026-05-16T14:30:00+05:00",
    "en_route_at": null,
    "arrived_at": null,
    "in_progress_at": null,
    "completed_at": null,
    "cancelled_at": null
  },

  // Quote audit trail (snapshot, never mutate after booking)
  "accepted_quote": {
    "quote_id": "QT-BUL2026-A8K3M2",
```

```
    "line_items": [...],
    "estimated_total_pkr": 2675,
    "estimated_range_pkr": [2260, 2890],
    "expires_at": "2026-05-16T14:45:00+05:00"
  },
  "actual_price_pkr": null,

  // Follow-up tracking
  "reminder_sent": false,
  "checkin_sent": false,
  "rating": null,
  "receipt_url": "https://storage.googleapis.com/bulao-receipts/BUL-...pdf",

  // Subcollection: bookings/{id}/dispute (only if status == "disputed")
  // "dispute": {
  //   "raised_at": "...",
  //   "user_complaint": "...",
  //   "service_checklist": [...],
  //   "rating": 2,
  //   "classification": "quality_complaint",
  //   "resolution": "partial_refund",
  //   "refund_pkr": 1250,
  //   "resolved_at": "..."
  // }
}

// Firestore indexes needed:
// 1. bookings: (status, scheduled_time) for Follow-up trigger query

// 2. bookings: (user_id, status) for listing a user's bookings
// Create via: gcloud firestore indexes create --project bulao-hackathon
```

APPENDIX D

Receipt PDF design specification.

Paste this into Antigravity with a prompt to implement `app/services/receipt_generator.py`. The receipt appears in the demo for 2 seconds — it must look designed.

```
// Receipt PDF design spec – backend/app/services/receipt_generator.py
// Use the reportlab library (already in pyproject.toml).
// The receipt must look DESIGNED, not generated. 8 visible zones:

ZONE 1 – Header (top 80px):
  Left: "Bulao." logotype in Helvetica Bold 22pt, Color(0xFF1a1d29)
  Right: "RASEED" in Helvetica Bold 9pt, Color(0xFFb8860b), letter-spaced
  Full-width hairline 0.8pt below header

ZONE 2 – Booking ID row:
  booking_id in monospace 11pt, SLATE
  "Confirm Ho Gaya" badge: GOLD bg, white text, rounded corners

ZONE 3 – Provider block:
  Provider name 14pt Bold INK | Rating stars | Years exp
  Neighborhood + city in 10pt SLATE

ZONE 4 – Service details:
  Two columns: Service Type (Urdu label) | Scheduled Time (natural Urdu)
  e.g. "AC Repair (Inverter)" | "Kal Subah 10 Baje"

ZONE 5 – Price breakdown (line items):
  Narrow table, 2 columns: Urdu label | PKR amount
  Last row = total, bold, larger font
  Hairline above total row

ZONE 6 – Provider earnings transparency:
  Italic 9pt MUTE: "Provider ko milenge PKR X platform fee ke baad"

ZONE 7 – QR code (bottom center):
  QR code of booking_id URL: https://bulao.app/booking/{booking_id}
  Caption: "Booking track karein"

ZONE 8 – Footer:
  "Bulao – Bolo, aur kaam ho jaye." centered, 8pt GOLD italic
  Cloud Storage URL where this PDF is hosted

PAGE: A5 portrait (148mm × 210mm), 12mm margins, PAPER background
```